

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

This section introduces the strategic solution and technical blueprint for the "Risk-Aversion Steward", addressing the problem of manual inventory management, capital lockup in dead stock, and reactive supply chain workflows.

1.1. Purpose:

The system analysis document highlights the technical scope, AI integration strategies, and design-related decisions behind the Risk-Aversion Steward development

So, the key element of system that has been covered are as followed:

1. Architecture:

- a. *It is a Client-Server application utilizing a modern Python (FastAPI) backend and a lightweight, responsive Vanilla JS/TailwindCSS frontend*

2. Data Flows:

- a. *How unstructured data (Customer Invoices in PDF/Word/Excel) flows through the Python extraction layer, gets processed by the LLM into structured JSON, updates the SQLite database, and triggers SMTP email communications to suppliers.*

3. Model Process:

- a. *The model showcases the end-to-end workflow of core features: predictive stockout alerts, AI-driven invoice outbound scanning, and generating actionable clearance strategies for stagnant inventory.*

4. Role of Reference:

- a. *This document acts as the definitive reference for developers and judges to understand the high-level architecture, fallback mechanisms, and AI prompt engineering used in this project.*

1.2. Background:

Small to Medium Enterprises (SMEs) currently rely on manual spreadsheet updates and visual shelf checks. When bulk orders arrive via invoices, workers manually type SKUs and quantities, a process prone to human error and massive delays.

1. Previous Version:

- a. *Inventory was managed using static tools (Excel/Paper). Restocking required manual calculation of sales velocity, and liquidation strategies were based on blind guessin.*

2. Changes in Major Architectural Components:

- a. *Transitioned to an AI-Native architecture where LLMs act as the bridge between unstructured offline documents and the structured SQL database.*

3. *New capabilities that are only introduced in this:*
- a. *AI Sales Scanner: Automated extraction of SKUs/Qty from invoices.*
 - b. *Automated Procurement: 1-click Auto-Order with SMTP email dispatch.*
 - c. *Smart Clearance Engine: AI-generated liquidation recommendations.*

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Warehouse / Store Manager	Uploads invoices, oversees daily stock levels, and confirms outbound/inbound operations.	Elimination of manual data entry; simple interface; clear warnings when stock falls below 15 days.
Business Owner / Founder	Reviews capital at risk, approves clearance strategies, and evaluates overall supply chain health.	Data-driven strategies to recover cash from dead stock; automated insights without needing technical skills.
Supplier	Receives automated purchase orders and cancellation notices.	Accurate, timely, and professionally formatted email notifications containing precise SKUs and quantities.
Development Team	Builds the AI integration, fallback routing, and frontend UI.	Clear API contracts, modular FastAPI structure, and robust error-handling for AI timeouts.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web-based Dashboard (Accessible via Desktop & Tablet)
Architecture	Client-Server Architecture (RESTful APIs with AI Service Layer)

Risk-Aversion Steward is structured with a lightweight Vanilla JS frontend communicating via REST API to a Python FastAPI backend. The backend manages local persistence using SQLite and acts as an orchestrator, connecting to external LLM APIs (ILMU/Z.ai) for reasoning and SMTP servers for automated communications.

2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic “AI Module”. The following Dependency diagram includes:

- The architecture integrates GLM as a core data-transformation and reasoning layer, rather than just a chatbot. It handles OCR-extracted text and converts it into strict JSON payloads for database operations.

2.1.3 Dependency Diagram

- Prompt Construction: Prompts are dynamically constructed in the backend. The backend queries the SQLite DB for current valid System SKUs and injects them into the prompt alongside the raw invoice text.*
- Context Window: The context contains a heavily truncated version of the invoice text (limited to the first 800-1000 characters) and a minified JSON array of valid SKUs to save tokens.*
- Response Parsing: The backend receives the LLM response, uses Regex (`re.search(r'[. *\\']')`) to strip away markdown/conversational text, parses the JSON, and standardizes the payload (e.g., forcing integers).*
- Token Limitations: Enforced locally using Python slicing (`raw_text[:800]`) before injecting into the prompt, preventing API timeouts and lowering token*

costs.

e. Major

API

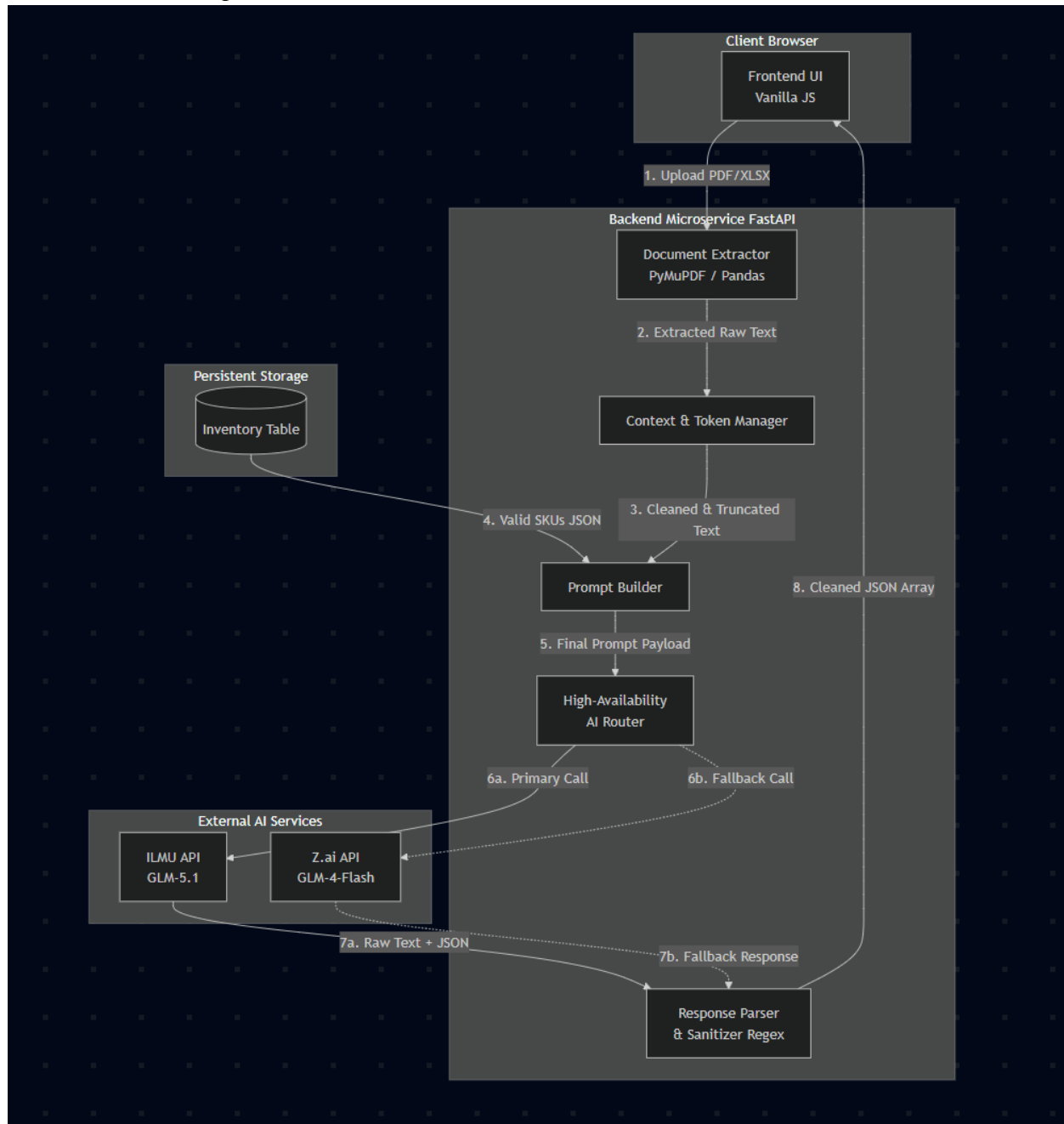
Calls: Frontend fetch() -> FastAPI

/api/invoice/analyze -> ILMU

API

/v1/chat/completions.

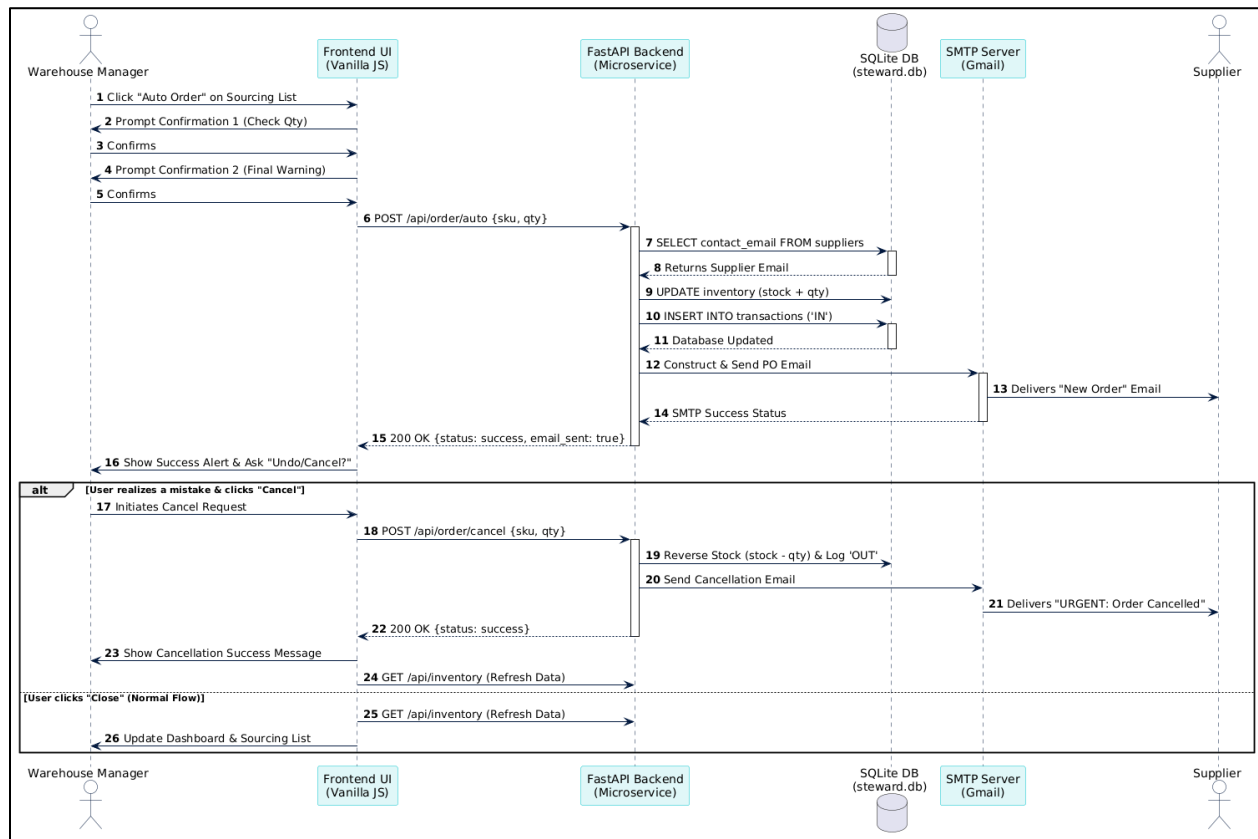
f. Component Interaction: FastAPI reads PDF via PyMuPDF, sends text to ILMU API, receives JSON, updates inventory table in SQLite, inserts transactions logs, and sends a response back to the JS Frontend to update the DOM.



2.2.2. Sequence Diagram

a. User clicks "Auto Order" on the Sourcing Tab.

- b. Frontend triggers a double-confirmation warning (Step 1 & Step 2).
- c. Frontend sends POST /api/order/auto with SKU and Qty.
- d. FastAPI Backend queries SQLite for Supplier Email.
- e. FastAPI Backend executes UPDATE inventory and INSERT transactions.
- f. FastAPI Backend triggers smtpplib to send an Order Email to the supplier.
- g. FastAPI Backend returns Success to Frontend.
- h. User is prompted with an optional "Undo/Cancel" mechanism.



2.2. Technological Stack

2.2.1. Frontend: HTML5, TailwindCSS (via CDN), Vanilla JavaScript, FontAwesome, Chart.js.

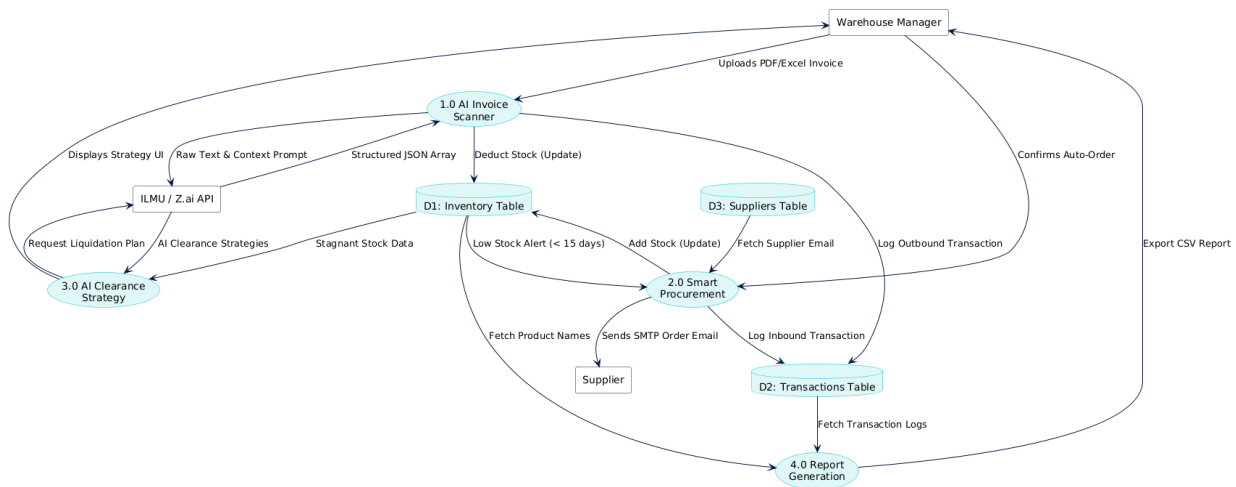
- 2.2.2. *Backend: Python 3.12.7, FastAPI, Uvicorn, PyMuPDF (PDF parsing), pandas (Excel parsing), python-docx (Word parsing), smtplib (Emailing).*
- 2.2.3. *Database: SQLite (Relational Database containing inventory, transactions, suppliers, and users).*
- 2.2.4. *Cloud/Deployment Google Cloud Platform (GCP).*

2.3. Key Data Flows

2.3.1. *Here, do show the how data are moving through the system and how its structure and stored*

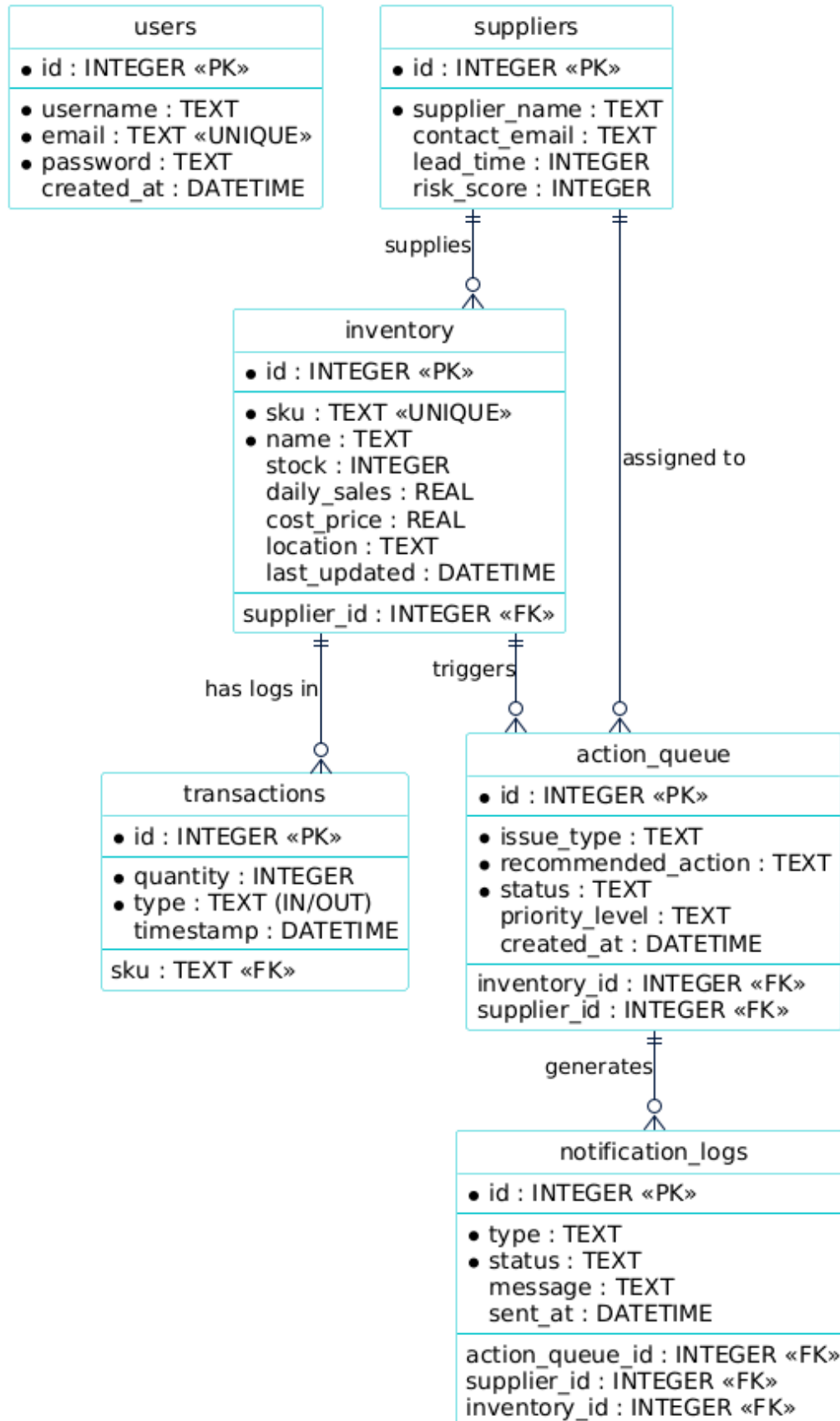
2.3.1.1. Data Flow Diagram (DFD)

- a. *Illustrate the data movement across the entire system that includes external entities (users), processes, external service and databases*
- b. *Show how proper laws are met (e.g. Black Hole)*
- c. *Inbound Data: User Uploads Invoice -> Python Extract Text -> Clean Non-ASCII -> LLM JSON Array -> UI Review -> Confirm -> DB Update.*
- d. *Outbound Data: DB detects stock < 15 days -> UI Sourcing Alert -> User Confirms -> DB Update -> SMTP Email dispatched to Supplier.*



2.3.2. Normalized Database Schema

- a. inventory (id, sku, name, stock, daily_sales, supplier_id)
- b. suppliers (id, supplier_name, contact_email)
- c. transactions (id, sku, quantity, type, timestamp)



3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.

#	Features	Description
1	AI Sales Scanner	Users upload PDF/Word/Excel invoices. System extracts text, cleans it, and uses AI to match items to system SKUs, preparing a bulk stock-deduction list.
2	Automated Sourcing	System auto-calculates 15-day stockout risks. Users can trigger "Auto Order," which updates the database and instantly emails the supplier a Purchase Order.
3	Order Cancellation	A fail-safe mechanism allowing users to revert accidental auto-orders, auto-adjusting DB stock and sending an urgent cancellation email.
4	AI Clearance Strategy	AI analyzes stagnant inventory (>45 days of stock) and generates customized liquidation strategies (e.g., BOGO, Flash Sales) with expected financial recovery metrics.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system.

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Reliability (AI)	The AI integration must not crash the app if the API provider goes down.	Implemented a 3-Tier High-Availability Strategy: ILMU API -> Z.ai API Fallback -> Local Smart Demo Fallback (Rule-based data generation).

Token Latency	The API calls to GLM for invoice scanning must not time out.	Implemented strict context truncation (<code>raw_text[:800]</code>) and extended the HTTP request timeout to 120s to accommodate peak LLM loads.
Data Integrity	Invoice parsing must map strictly to existing database items, preventing "hallucinated" SKUs.	The prompt dynamically injects a minified JSON array of valid System SKUs and strictly commands the AI to match against them.

3.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for “Grand Vision” but not part of this UMHackathon 2026. [e.g. below]

- a. Direct integration with Payment Gateways or ERPs (e.g., SAP, Oracle).*
- b. Visual OCR processing for flattened, unselectable image-based receipts (JPG/PNG).*
- c. Multi-user concurrent editing and Role-Based Access Control (RBAC).*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

- 4.1.1. *High-Availability Fallback: We evaluate API health during every call. If ILMU-GLM-5.1 fails or exceeds the 15-second soft timeout, the system seamlessly redirects the prompt to Z.ai (GLM-4-Flash).*
- 4.1.2. *Smart Demo Fallback (Emergency Strategy): If no internet connection is available or all AI APIs are exhausted, the system triggers a localized simulation module. It randomly samples local DB SKUs to simulate a successful invoice extraction, guaranteeing the UI workflow never breaks during demonstrations.*

4.1.3. **Priority Matrix:**

4.1.3.1. Clearly define the Priority Matrix and what are the trigger conditions and what actions will be taken for your project.

Do Next (High Importance, Low Urgency) • AI Clearance Strategies: Generating tailored liquidation plans (e.g., Flash Sales) for stagnant inventory (> 45 days old) to recover frozen capital. • AI Sales Scanner: Processing bulk PDF/Excel invoices into structured JSON. While highly important for automation, slight processing delays (e.g., waiting 5-10 seconds for AI) are acceptable. • Future Scaling: Planning the database migration from local SQLite to Google Cloud SQL (PostgreSQL).	Do Now (High Importance, High Urgency) • System Outages: Fixing Backend (FastAPI) or Database (SQLite) crashes. • Critical Alerts: Processing items with < 15 days of stock (High Stockout Risk) and triggering the automated Restock Email to suppliers. • AI Fail-safes: Triggering the "Smart Demo Fallback" immediately if the primary ILMU API times out, ensuring the system never freezes during a live operation.
Do Never / Drop (Low Importance, Low Urgency) • Full ERP Integration: Integrating deeply with heavy legacy systems like SAP or Oracle (out of scope for this hackathon). • Custom OCR Training: Training an AI Vision model from scratch to read blurry JPEG receipts (we rely on digital PDF/XLSX text extraction instead to save time and resources). • Multi-User Role Systems (RBAC): Building complex admin/staff permission levels for a prototype designed to showcase core AI inventory capabilities.	Do Last (Low Importance, High Urgency) • UI/UX Glitches: Fixing TailwindCSS alignment issues, mobile responsiveness, or button hover states. • Report Formatting: Minor adjustments to the Exported CSV headers or timestamp formats. • Action: These are pushed to the end of the sprint or delegated to the UI/UX Lead to fix only after core API functionalities are 100% stable

4.2. **Monitoring:**

4.2.1. Backend terminal executes continuous debug tracing ([DEBUG]) indicating file ingestion status, text truncation previews, SMTP email dispatch status, and stack traces via `traceback.print_exc()` to instantly identify logical breaks.

4.3. **Assumptions:** State key operational & environmental conditions that you assumed to be valid while development of system and deployment.

- a. Suppliers have valid, reachable email addresses configured in the database.
- b. Uploaded invoices are digitally generated documents containing readable text layers, not flattened scanned images.
- c. The host machine running the backend has no firewall blocking outbound SMTP (Port 587) connections.

4.4. External Dependencies: Do list all the external tools that are applied to your system to function.

Tools	Purpose	Risks
ILMU API / Z.ai API	Core LLM reasoning engine to parse unstructured text and generate JSON/Strategies.	High API traffic latency. Mitigation: Truncated inputs & 3-tier fallback architecture.
PyMuPDF / Pandas	Extracts raw text from uploaded .pdf and .xlsx documents.	Encountering hidden control characters. Mitigation: RegEx sanitization implemented.
SMTP Server	Dispatches automated Purchase Orders and Cancellation emails.	Authentication blocks. Mitigation: Try-except wrapper logs emails locally if auth fails.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

Day 1-3: Database schema design (SQLite) and Vanilla JS + TailwindCSS frontend scaffolding.

Day 4-6: FastAPI backend routing, CRUD operations, and standard inventory logic.

Day 7-8: LLM prompt engineering, integration of PyMuPDF/Pandas, and AI Sales Scanner development.

Day 9-10: SMTP Email integration, 3-tier Fallback implementation, error handling, final UI polishing and deployment.

5.2. Team Members Role:

Lee Jun Kiat – Project Manager & Technical Writer (System Documentation & QA)

Ding Jia Jun – Backend Lead (FastAPI Microservices & Database Architecture)

Thim Wong – Frontend & UI/UX Lead (Client-Side Development)

Hoo Jin Yi – AI Solutions Engineer (LLM Integration & Prompt Engineering)

5.3. Recommendations:

- a. *For Production Scaling: Migrate the database from SQLite to PostgreSQL.*
- b. *For AI Performance: Implement an asynchronous task queue (like Celery/Redis) so users don't have to wait on the loading screen while the AI processes massive 50-page invoices in the background.*
- c. *For generalization: Implement the whatsapp and telegram api key to send order to suppliers.*